Single Machine Scheduling

Minimizing Maximum Tardiness

CH. Taraka Abhinav - EE25BTECH11016

1. The Problem Statement

Question (2007 PI)

Four jobs have to be sequenced on a single facility, with the objective of minimizing the maximum tardiness: $\max_i |\text{Completion time}_i - \text{Due date}_i|$.

The jobs have due dates and processing times as follows:

Job	Due date (days)	Processing time (days)
P	5	2
Q	6	10
R	3	3
S	7	4

The last job that should be taken up is:

a) P b) Q c) R d) S

2. Understanding the Concepts

What are the variables?

- **Processing Time (p_i):** How long it takes to finish the job.
- **Due Date (d_i):** The deadline promised to the customer.
- **Completion Time (C_i):** The actual day the job finishes.

What is Tardiness / Lateness?

Lateness is simply: Completion Time − Due Date $(C_i - d_i)$.

If a job finishes *after* its due date, it generates a penalty. Our goal is to arrange these jobs so that the **single worst delay** (Maximum Tardiness) is as small as physically possible.

3. Common Scheduling Rules

To sequence jobs on a single machine, we use standard dispatching rules:

- **SPT (Shortest Processing Time):** Sort by smallest p_i.
  Objective: Minimizes average completion time / flow time.
- **LST (Least Slack Time):** Sort by smallest $(d_i - p_i)$.
  Objective: Prioritizes jobs with the least buffer time.
- **EDD (Earliest Due Date):** Sort by smallest d_i.
  Objective: **Minimizes Maximum Tardiness ($T_{\max}$)**.

Which rule to choose?

Since our problem explicitly asks to minimize the **maximum tardiness**, we must use the **EDD Rule**.

4. Applying the EDD Rule

Let's extract the due dates from our original table and sort them from smallest (earliest) to largest (latest).

Original Job	Due Date (d_i)	$\rightarrow$	Sorted Order (EDD)
P	5		**2nd**
Q	6		**3rd**
R	**3** (Earliest)		**1st**
S	7 (Latest)		**4th**

Based purely on the EDD rule, the optimal processing sequence is:

$$\mathbf{R} \rightarrow \mathbf{P} \rightarrow \mathbf{Q} \rightarrow \mathbf{S}$$

5. Verifying the Schedule (Proof)

Let's calculate the exact completion times to prove this works:

Sequence	Proc. Time (p_i)	Completion Time (C_i)	Due Date	Tardiness ($\lvert C_i - d_i \rvert$)
1. Job R	3	$0 + 3 = \mathbf{3}$	3	$\max(0, 3 - 3) = \mathbf{0}$
2. Job P	2	$3 + 2 = \mathbf{5}$	5	$\max(0, 5 - 5) = \mathbf{0}$
3. Job Q	10	$5 + 10 = \mathbf{15}$	6	$\max(0, 15 - 6) = \mathbf{9}$
4. Job S	4	$15 + 4 = \mathbf{19}$	7	$\max(0, 19 - 7) = \mathbf{12}$

- The maximum tardiness in this optimal sequence is 12 days.
- *Note: If we placed Job Q last, Job Q would finish on Day 19 (Due Day 6), creating a worse maximum penalty of 13!*

6. Numerical Proof: What if we used SPT?

Let's test the **SPT (Shortest Processing Time)** rule, an intuitive alternative. We sort jobs by smallest p_i:

SPT Sequence: P (2) $\rightarrow$ R (3) $\rightarrow$ S (4) $\rightarrow$ Q (10)

Sequence (SPT)	Proc. Time	Completion Time	Due Date	Tardiness
1. Job P	2	$0 + 2 = \mathbf{2}$	5	$\max(0, 2 - 5) = \mathbf{0}$
2. Job R	3	$2 + 3 = \mathbf{5}$	3	$\max(0, 5 - 3) = \mathbf{2}$
3. Job S	4	$5 + 4 = \mathbf{9}$	7	$\max(0, 9 - 7) = \mathbf{2}$
4. Job Q	10	$9 + 10 = \mathbf{19}$	6	$\max(0, 19 - 6) = \mathbf{13}$

Result: The maximum tardiness under SPT is **13 days** (caused by Job Q).

7. Numerical Proof: What if we used LST?

Let's test the **LST (Least Slack Time)** rule. Slack = Due Date - Processing Time ($d_i - p_i$).

Slack Calc: P(5-2=3), Q(6-10=-4), R(3-3=0), S(7-4=3)
LST Sequence: Q (-4) $\rightarrow$ R (0) $\rightarrow$ P (3) $\rightarrow$ S (3)

Sequence (LST)	Proc. Time	Completion Time	Due Date	Tardiness
1. Job Q	10	$0 + 10 = \mathbf{10}$	6	$\max(0, 10 - 6) = \mathbf{4}$
2. Job R	3	$10 + 3 = \mathbf{13}$	3	$\max(0, 13 - 3) = \mathbf{10}$
3. Job P	2	$13 + 2 = \mathbf{15}$	5	$\max(0, 15 - 5) = \mathbf{10}$
4. Job S	4	$15 + 4 = \mathbf{19}$	7	$\max(0, 19 - 7) = \mathbf{12}$

Result: The maximum tardiness is **12 days**, but notice that *every single job* is penalized!

8. Comparison: EDD vs SPT vs LST

Rule & Sequence	Tardiness Profile	Max Tardiness ($T_{\max}$)
EDD (R-P-Q-S)	{0, 0, 9, **12**}	**12** (Optimal)
SPT (P-R-S-Q)	{0, 2, 2, **13**}	**13**
LST (Q-R-P-S)	{4, 10, 10, **12**}	**12**

Conclusion:

- **SPT** pushes the long job (Q) to the end, causing a peak penalty spike of 13.
- **LST** ties the peak penalty of 12, but creates a *cascading delay* where all jobs are severely late (Sum of penalties = 36 vs EDD's 21).
- **EDD** mathematically guarantees the lowest possible maximum penalty while ensuring early jobs finish on time!

Final Conclusion

The problem asks for the **last job** that should be taken up in the optimal sequence.

Optimal Sequence Result

$$\mathbf{R} \rightarrow \mathbf{P} \rightarrow \mathbf{Q} \rightarrow \mathbf{S}$$

The last job in the sequence is **Job S**.

Correct Option: (d) S

Decision Variables

We need to determine:

- **Completion times**

$$C = \begin{bmatrix} C_1 \\ C_2 \\ C_3 \\ C_4 \end{bmatrix}$$

- **Order of jobs**

$$X_{ij} = \begin{cases} 1 & \text{if job } i \text{ is before } j \\ 0 & \text{otherwise} \end{cases}$$

- **Maximum tardiness**

$$T_{\max}$$

Modeling Idea

Core idea:

- Use binary variables X_{ij} to represent ordering
- Enforce **no overlap** using Big-M constraints
- Compute completion times automatically
- Minimize the worst delay ($T_{\max}$)

Why this works:

- The solver tries all feasible sequences implicitly
- Returns the optimal schedule directly

Mathematical Formulation

Objective:

$$\min \ T_{\max}$$

Basic constraints:

$$C \geq p$$

$$X + X^T = \mathbf{1}\mathbf{1}^T - I$$

$$\mathrm{diag}(X) = 0$$

Sequencing and Tardiness Constraints

No-overlap (Big-M):

$$C_i + p_j \leq C_j + M(1 - X_{ij}), \quad i \neq j$$

Tardiness definition:

$$T_{\max} \geq C - d$$

$$T_{\max} \geq 0$$

Meaning:

- Ensures correct job order
- Captures maximum delay among all jobs

Optimal Solution

Solver Output:

$$C = [5,\ 15,\ 3,\ 19]$$

$$T_{\max} = 12$$

Ordering matrix (interpreted):

- R precedes all jobs
- P precedes Q, S
- Q precedes S

Final Schedule

Optimal job sequence:

$$R \to P \to Q \to S$$

Key observation:

- Sequence is obtained directly from optimization
- No heuristic or rule required

Conclusion:

- MILP model guarantees optimal solution
- CVXPY computes it efficiently

Python: Data Setup & Analysis

```
import matplotlib.pyplot as plt
import numpy as np

# Job Data
jobs = {
    'P': {'p': 2, 'd': 5},
    'Q': {'p': 10, 'd': 6},
    'R': {'p': 3, 'd': 3},
    'S': {'p': 4, 'd': 7}
}

# Sequences
seq_edd = ['R', 'P', 'Q', 'S']
seq_spt = ['P', 'R', 'S', 'Q']
seq_lst = ['Q', 'R', 'S', 'P']

def analyze_sequence(seq):
    current_time = 0
    gantt = []
    abs_lateness = {}
    actual_lateness = {}
    cumulative_times = [0]
```

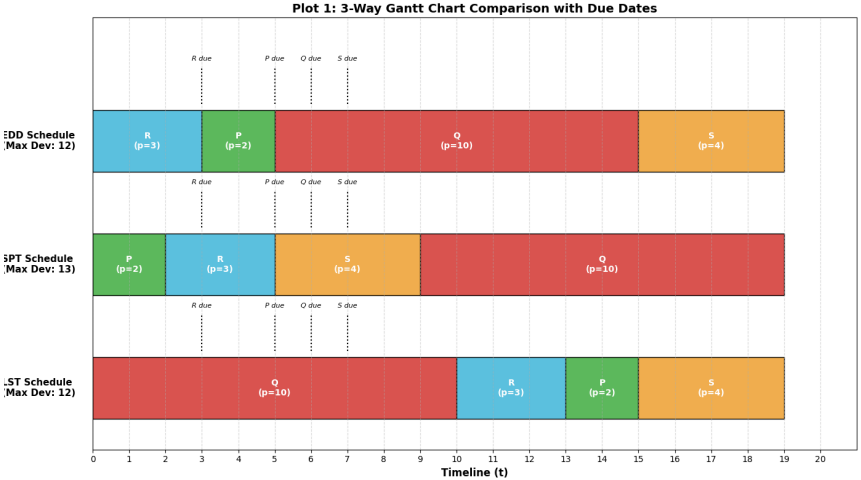
Python: Data Setup & Analysis

```
    for job in seq:
        p = jobs[job]['p']
        d = jobs[job]['d']
        start = current_time
        current_time += p

        gantt.append((job, start, p, d))
        abs_lateness[job] = abs(current_time - d)
        actual_lateness[job] = current_time - d
        cumulative_times.append(current_time)

    return gantt, abs_lateness, actual_lateness, cumulative_times
```

Python: Gantt Chart Generation

```
gantt_edd, _, _, _ = analyze_sequence(seq_edd)
gantt_spt, _, _, _ = analyze_sequence(seq_spt)
gantt_lst, _, _, _ = analyze_sequence(seq_lst)

colors = {'P': '#2ca02c', 'Q': '#d62728',
         'R': '#1f77b4', 'S': '#ff7f0e'}

fig1, ax1 = plt.subplots(figsize=(12, 8))

def draw_gantt_row(ax, gantt_data, y_base):
    for job, start, p, d in gantt_data:
        ax.broken_barh([(start, p)], (y_base, 6))
        ax.text(start + p/2, y_base + 3, job)

draw_gantt_row(ax1, gantt_lst, 5)
draw_gantt_row(ax1, gantt_spt, 20)
draw_gantt_row(ax1, gantt_edd, 35)

plt.savefig("G.png")
```

Gantt Chart Comparison

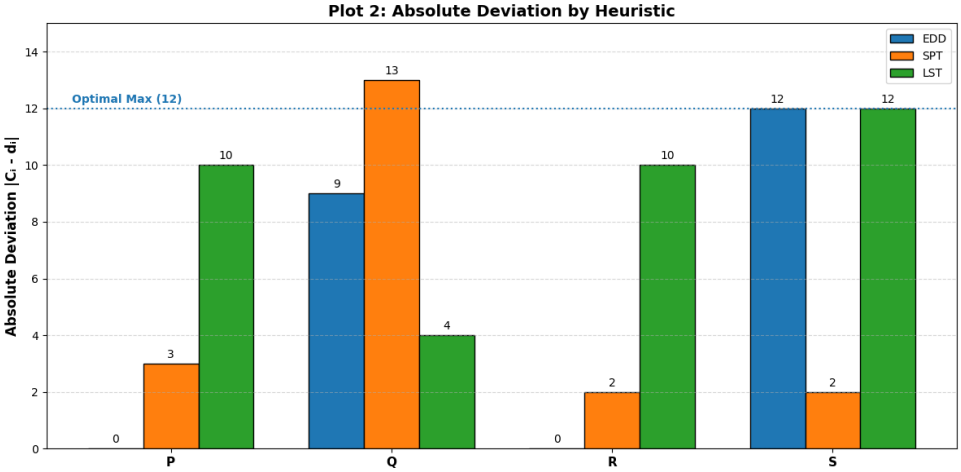


Observation:

Python: Absolute Deviation

```
job_labels = ['P', 'Q', 'R', 'S']
x = np.arange(len(job_labels))
width = 0.25

_, abs_lat_edd, _, _ = analyze_sequence(seq_edd)
_, abs_lat_spt, _, _ = analyze_sequence(seq_spt)
_, abs_lat_lst, _, _ = analyze_sequence(seq_lst)

edd_vals = [abs_lat_edd[j] for j in job_labels]
spt_vals = [abs_lat_spt[j] for j in job_labels]
lst_vals = [abs_lat_lst[j] for j in job_labels]

fig2, ax2 = plt.subplots()
ax2.bar(x - width, edd_vals, width, label='EDD')
ax2.bar(x, spt_vals, width, label='SPT')
ax2.bar(x + width, lst_vals, width, label='LST')

plt.savefig("A.png")
```

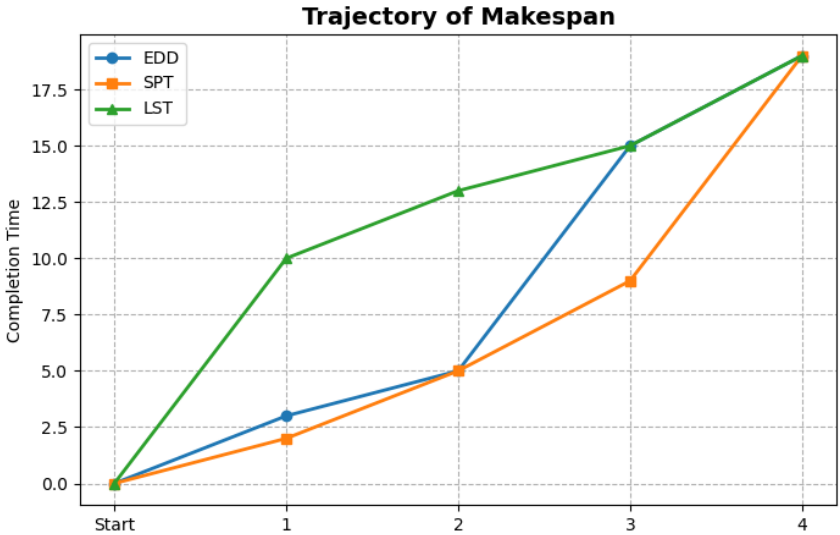
Absolute Deviation Plot



Python: Completion Time Trajectory

```
_, _, _, cum_edd = analyze_sequence(seq_edd)
_, _, _, cum_spt = analyze_sequence(seq_spt)
_, _, _, cum_lst = analyze_sequence(seq_lst)

steps = [0, 1, 2, 3, 4]

fig4, ax4 = plt.subplots()
ax4.step(steps, cum_edd, label='EDD', where='post')
ax4.step(steps, cum_spt, label='SPT', where='post')
ax4.step(steps, cum_lst, label='LST', where='post')

ax4.legend()
plt.savefig("T.png")
```

Completion Time Trajectory



CVXPY Implementation

```
import cvxpy as cp
import numpy as np

p = np.array([2,10,3,4])
d = np.array([5,6,3,7])
n = 4
M = 100

C = cp.Variable(n)
Tmax = cp.Variable()
X = cp.Variable((n,n), boolean=True)

constraints = [
    cp.diag(X) == 0,
    X + X.T == np.ones((n,n)) - np.eye(n),
    C >= p,
    Tmax >= C - d,
    Tmax >= 0
]
```

CVXPY Implementation

```
for i in range(n):
    for j in range(n):
        if i != j:
            constraints += [
                C[i] + p[j] <= C[j] + M*(1 - X[i,j])
            ]

prob = cp.Problem(cp.Minimize(Tmax), constraints)
prob.solve(solver=cp.ECOS_BB)
```

CVXPY Solver[ECOS] Output

```
Status: optimal
Tmax = 12.0
C = [ 5. 15.  3. 19.]
X =
 [[0. 1. 0. 1.]
 [0. 0. 0. 1.]
 [1. 1. 0. 1.]
 [0. 0. 0. 0.]]
Sequence: R -> P -> Q -> S
```